

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0515

Slot B

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A

Case Study

Code of Conduct:

I _Harshavardhan. RV(192472163)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Harshavardhan. RV

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1 Preprocessing and Feature Detection

A vision system processes images captured under varying lighting conditions. Without preprocessing, feature detection results are inconsistent. Analyze the issue and explain how preprocessing improves feature detection performance.

ANS: Preprocessing and Feature Detection

Introduction

In a computer vision system, images may be captured under different lighting conditions, such as bright sunlight, low light, shadows, or uneven illumination. These changes can make feature detection inconsistent. Image preprocessing is used to improve image quality and make important features such as edges, corners, and textures easier to detect.

1. Problem Caused by Varying Lighting

- Bright lighting can cause overexposed regions.
- Low lighting can hide important features.
- Shadows can create unwanted edges.
- Changes in brightness and contrast affect pixel values.
- Noise may increase in dark images.
- As a result, the same object may produce different features in different images.

2. Need for Preprocessing

Preprocessing prepares the image for accurate feature detection. It:

- Reduces unwanted noise.
- Improves contrast and brightness.
- Removes illumination variations.
- Enhances important structures.
- Makes features more stable and repeatable.

3. Important Preprocessing Techniques

a) Grayscale Conversion

Converts a color image into grayscale. It reduces the image complexity and focuses on intensity information.

b) Noise Removal

Gaussian or median filters can remove random noise while preserving important image structures.

c) Histogram Equalization

Improves the contrast of an image by redistributing pixel intensity values. It is useful for images with poor contrast.

d) CLAHE

Contrast Limited Adaptive Histogram Equalization improves **local contrast** and is especially useful when lighting is uneven.

e) Image Normalization

Adjusts pixel intensity values to a common range. This reduces differences caused by varying illumination.

f) Sharpening

Enhances edges and fine details, making them easier for feature detectors to identify.

4. Feature Detection After Preprocessing

After preprocessing, feature detection algorithms such as Harris Corner, SIFT, SURF, ORB, or FAST can detect features more reliably.

For example:

Original Image → Preprocessing → Enhanced Image → Feature Detection → Stable Features

5. Example

Consider a building photographed during daylight and evening.

Without preprocessing:

- Daylight image has high brightness.
- Evening image has low brightness.
- Different corners and edges may be detected.

With preprocessing:

- Brightness is normalized.
- Contrast is enhanced.
- Noise is reduced.
- Important corners and edges become clearer.
- Similar features can be detected in both images.

6. Benefits

- Improves feature detection accuracy.
- Reduces false feature detection.
- Provides better edge and corner detection.
- Improves feature matching between images.
- Makes the system more robust to lighting changes.
- Improves the overall performance of computer vision applications.

Conclusion

Preprocessing is an essential step in feature detection when images are captured under varying lighting conditions. Techniques such as grayscale conversion, noise filtering, histogram equalization, CLAHE, normalization, and sharpening improve image quality and reduce illumination effects. Therefore, preprocessing produces more accurate, stable, and consistent features, improving the reliability of the overall vision system.

2. Edge vs Corner Detection

In an image containing both straight edges and textured regions, a system uses edge detection but fails to identify key points for matching. Analyze why edge detection is insufficient and justify the use of corner detection in this case.

Edge vs Corner Detection

Introduction

In computer vision, edge detection and corner detection are used to identify important structures in an image. Edge detection finds boundaries, while corner detection identifies points where two or more edges meet. When an image contains straight edges and textured regions, using only edge detection may not provide enough unique points for accurate image matching.

1. Why Edge Detection is Insufficient

Edge detection identifies locations where **the intensity of pixels changes significantly**.

However:

- A straight edge mainly provides information in one direction.
- Points along the same edge may look very similar.
- It is difficult to determine the exact position of a point along a long straight edge.
- Edges do not always provide unique and distinctive key points.
- Textured regions may contain many small edges, but these edges may not be suitable for reliable matching.

Therefore, edge detection is useful for finding object boundaries but is not sufficient for finding **distinctive key points**.

2. Need for Corner Detection

A corner is a point where the intensity changes significantly in two or more directions. Corners provide more unique information than straight edges because their intensity changes in multiple directions.

3. Advantages of Corner Detection

- Detects **distinctive key points**.
- Provides better localization of important points.
- Works well for image matching and tracking.
- Corners are usually more stable than points on straight edges.
- Helps identify important structures in textured regions.
- Provides features that can be matched between different images.

4. Example

Consider an image of a **building**.

Edge detection can identify:

- Walls
- Windows

- Roof boundaries
- Straight lines

But it may not identify unique points for matching.

Corner detection can identify:

- Corners of windows
- Corners of doors
- Intersections of walls
- Junctions of edges

These points are more distinctive and can be used as **key points for image matching**.

5. Edge vs Corner Detection

Edge Detection	Corner Detection
Detects boundaries	Detects distinctive points
Usually changes in one direction	Changes in two or more directions
Produces lines/curves	Produces key points
Less suitable for point matching	Highly suitable for point matching
Used for object boundaries	Used for matching, tracking and recognition

6. Common Corner Detection Methods

Some commonly used methods are:

- Harris Corner Detector
- Shi-Tomasi Corner Detector
- FAST
- SIFT/ORB keypoint detection

Conclusion

Edge detection alone is insufficient because it mainly identifies continuous boundaries and straight lines, which may not provide unique points for matching. Corner detection identifies points where intensity changes in multiple directions, producing distinctive and well-localized key points. Therefore, for an image containing edges and textured regions, corner detection is more suitable for image matching, tracking, and feature extraction.

3. Hough Transform in Noisy Images

A system uses Hough Transform to detect lines in noisy images but produces inaccurate results.

Analyze the effect of noise on detection and suggest improvements based on preprocessing or parameter selection.

Hough Transform in Noisy Images

Introduction

The Hough Transform is a popular image processing technique used to detect geometric shapes such as lines and circles. It works by converting image points into a parameter space and finding parameters where many points agree. However, noise in an image can produce unwanted points, resulting in inaccurate line detection.

1. Effect of Noise on Hough Transform

Noise introduces unwanted pixel variations and false edges in an image.

The effects are:

- Noise creates false edge points.
- These points can vote for incorrect lines.
- The accumulator receives unwanted votes.
- Weak or broken lines may not receive enough votes.
- Multiple false lines may be detected.
- The detected line position and angle may be inaccurate.
- Detection becomes less reliable as noise increases.

2. Why Inaccurate Results Occur

The Hough Transform depends on edge points.

Noisy Image → False Edges → Incorrect Votes → False Lines

For example, if random noise creates several isolated bright pixels, the Hough Transform may interpret these pixels as parts of a line.

3. Preprocessing Improvements

Before applying the Hough Transform, preprocessing can reduce noise.

a) Gaussian Filtering

Smooths the image and reduces random noise.

b) Median Filtering

Useful for removing salt-and-pepper noise while preserving edges.

c) Thresholding

Removes weak or unwanted pixels before line detection.

d) Edge Detection

Using methods such as **Canny Edge Detection** produces cleaner edge maps for the Hough Transform.

e) Morphological Operations

Opening and closing can remove small noise and connect broken edges.

4. Parameter Selection

Proper Hough Transform parameters can improve detection.

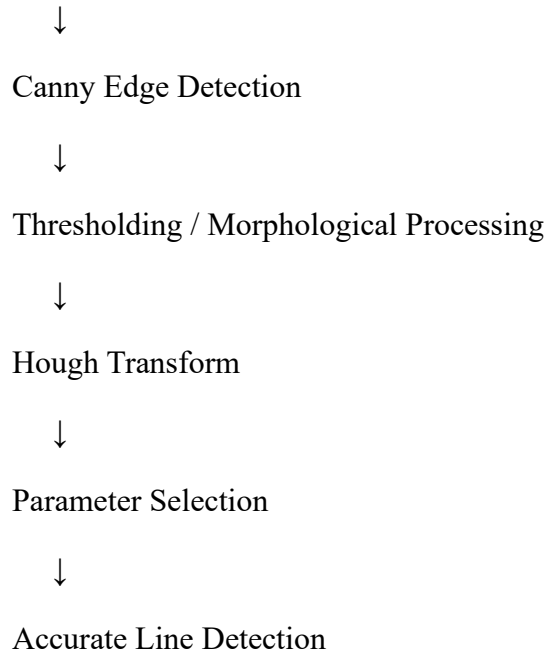
- **Threshold:** Increase the voting threshold to reject weak false lines.
- **Resolution:** Select suitable distance and angle resolutions.
- **Minimum line length:** Ignore very short lines caused by noise.
- **Maximum line gap:** Allow small gaps while connecting genuine line segments.
- **Angle range:** Restrict detection to expected line orientations when possible.

5. Improved Process

Noisy Image



Noise Filtering



6. Example

Suppose an image contains a road with lane markings and considerable camera noise.

Without preprocessing:

- Noise creates false edges.
- Hough Transform detects several incorrect lines.
- Lane lines may be incorrectly positioned.

With preprocessing:

- Gaussian filtering removes noise.
- Canny detects strong edges.
- A suitable Hough threshold removes weak votes.
- Minimum line length eliminates short false lines.
- The actual lane lines are detected more accurately.

7. Benefits of Improvements

- Reduces false line detection.
- Improves line position accuracy.

- Removes unwanted noise.
- Detects weak but meaningful lines more reliably.
- Improves the overall robustness of the system.

Conclusion

Noise can significantly reduce the accuracy of the Hough Transform by creating false edges and incorrect votes in the parameter space. The problem can be reduced by applying Gaussian/median filtering, Canny edge detection, thresholding, and morphological operations before the Hough Transform. Proper selection of parameters such as voting threshold, minimum line length, maximum line gap, and angle resolution further improves line detection accuracy.

4. Feature Matching under Transformations

Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale invariant feature matching works effectively. Analyze why scale invariant methods perform better in this scenario.

Feature Matching Under Transformations

Introduction

Feature matching is used to identify the same object or scene in two different images. However, when images are captured at different scales and orientations, traditional feature matching may fail because the appearance and position of features change. Scale-invariant feature matching methods are designed to handle these transformations effectively.

1. Problem with Traditional Feature Matching

Traditional methods compare image features based on their original appearance or position.

When an object is changed in scale or orientation:

- The size of the feature changes.
- The orientation of the feature changes.
- Pixel patterns may appear different.
- The location of key points changes.
- Direct comparison becomes difficult.

- Correct matches may be missed.

For example, a small image of a car and a larger rotated image of the same car may have very different pixel arrangements.

2. Scale and Rotation Changes

Scale change:

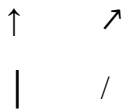
Small Object → Large Object



The same feature appears at different sizes.

Rotation change:

Original → Rotated



The orientation of the same feature changes.

Traditional matching may treat these as different features.

3. Scale-Invariant Feature Matching

Scale-invariant methods such as **SIFT** are designed to detect features that remain recognizable even when the image is resized or rotated.

They generally perform the following steps:

1. **Detect key points** in the image.
2. Determine the **scale** at which each key point is most stable.
3. Determine the **orientation** of the key point.
4. Create a feature **descriptor** representing the local region.
5. Compare descriptors between the two images.

Because the descriptor is based on normalized local information, the same feature can be matched even after scale or rotation changes.

4. Why Scale-Invariant Methods Perform Better

- **Scale invariance:** Detects the same feature at different sizes.
- **Rotation invariance:** Matches features even when the object is rotated.
- **Robust descriptors:** Descriptors capture local structure rather than exact pixel positions.
- **Better key-point detection:** Stable points are selected from different image scales.
- **Reliable matching:** Similar features receive similar descriptors despite transformations.

5. Example

Consider two images of a logo:

- Image 1: Normal size and orientation.
- Image 2: Larger and rotated by 45°.

Traditional matching:

- Feature positions and sizes are different.
- Many features fail to match.

Scale-invariant matching:

- Detects the same key points.
- Handles the size difference.
- Adjusts for orientation.
- Produces more correct matches.

6. Comparison

Traditional Matching	Scale-Invariant Matching
Sensitive to scale changes	Handles scale changes
Sensitive to rotation	Handles rotation
Depends more on pixel arrangement	Uses robust feature descriptors
Fewer correct matches	More reliable matches
Suitable for simple conditions	Suitable for real-world transformations

7. Applications

Scale-invariant feature matching is widely used in:

- Object recognition
- Image stitching
- Panorama creation
- Object tracking
- Augmented reality
- Image retrieval
- Robotics and navigation

Conclusion

Traditional feature matching performs poorly when images have significant scale and orientation changes because the size, position, and appearance of features change. Scale-invariant methods such as SIFT detect stable key points across different scales and assign orientations to them. Their robust descriptors allow the same features to be matched even after scaling and rotation, making them more reliable for real-world computer vision applications.

5. PCA for Feature Optimization

A system extracts a large number of features, leading to high computational cost and redundancy. Analyze how Principal Component Analysis helps in reducing dimensionality and improving efficiency.

PCA for Feature Optimization

Introduction

In computer vision and machine learning, a system may extract a large number of features from images. Many of these features may contain similar or redundant information. Processing a large feature set increases memory usage, computational cost, and processing time. Principal Component Analysis (PCA) is used to reduce the number of features while preserving the most important information.

1. Problem with Large Number of Features

Extracting many features can cause:

- High computational cost.

- Increased memory usage.
- Longer training and prediction time.
- Redundant or correlated features.
- Difficulty in visualizing the data.
- Reduced efficiency of machine learning algorithms.

For example, an image may initially have **100 features**, but many of them may provide similar information.

2. What is PCA?

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms a large set of correlated features into a smaller set of **uncorrelated principal components**.

The principal components are arranged according to the amount of information or **variance** they preserve.

3. Working of PCA

The main steps are:

Step 1: Collect features

Create a feature matrix from the extracted image features.

Step 2: Standardize the data

Scale the features so that features with large numerical values do not dominate.

Step 3: Calculate covariance matrix

Find relationships between different features.

Step 4: Calculate eigenvalues and eigenvectors

These identify the important directions in the data.

Step 5: Select principal components

Choose the components with the highest eigenvalues because they contain the most information.

Step 6: Transform the data

Project the original high-dimensional features onto the selected components.

4. Example

Suppose a system extracts **100 features** from an image.

100 Original Features



PCA



20 Principal Components



Machine Learning / Classification

If the first 20 components preserve most of the useful information, the remaining 80 features can be removed.

5. How PCA Improves Efficiency

- **Reduces dimensionality:** Fewer features need to be processed.
- **Reduces computation:** Algorithms operate on smaller datasets.
- **Reduces memory usage:** Less storage is required.
- **Removes redundancy:** Correlated information is combined.
- **Speeds up training:** Machine learning models train faster.
- **Improves visualization:** Data can be represented in 2D or 3D.
- **Reduces noise:** Less important variations can be discarded.

6. PCA Process

Original High-Dimensional Features



Standardization



Covariance Matrix



Eigenvalues & Eigenvectors



Select Important Components



Reduced Feature Dataset



Faster Processing

7. Advantages of PCA

- Simple and widely used dimensionality reduction method.
- Preserves maximum variance using fewer components.
- Reduces redundant features.
- Improves computational efficiency.
- Can improve model performance when irrelevant features are removed.

8. Limitation

PCA may reduce interpretability because the new principal components are combinations of the original features. Also, PCA mainly captures linear relationships in the data.

Conclusion

PCA helps optimize feature extraction by converting a large number of correlated features into a smaller set of important principal components. This reduces computational cost, memory usage, processing time, and redundancy while retaining most of the important information. Therefore, PCA is an effective technique for improving the efficiency of computer vision and machine learning systems.